

Software Requirements Specification Outline

TUI RAG Chatbot with Toggleable Agent Features

1. Project Overview

1.1 Product Name

Working name: **SoulForge TUI**

1.2 Purpose

SoulForge TUI is a local-first terminal user interface chatbot that runs GGUF models through `llama-cpp-python`, supports CUDA GPU acceleration, uses `SOUL.md` for persona control, and adds modular RAG, persistent memory, reusable skills, task tracking, and toggleable agent features.

The system should evolve from a simple local chatbot into a stable, user-personalized assistant that can retrieve local knowledge, remember useful context, crystallize repeated workflows into reusable skills, and keep its memory and skills clean over time.

1.3 Intended Users

- Local LLM users
- Developers
- Cybersecurity students and researchers
- Writers and roleplay users
- Users who want a private, local agent system
- Users who want RAG and memory without depending on cloud services

1.4 Product Vision

The application should behave less like a fragile experiment and more like a stable local product. It should be easy to start, easy to configure, and hard to break.

Core design principle:

```
SOUL.md controls personality.  
memory.md controls durable context.  
docs/ controls knowledge.  
skills/ controls reusable workflows.  
toggles control agent behavior.
```

2. Current Baseline

2.1 Existing System

The current project already includes:

- WSL Ubuntu setup
- Python virtual environment
- llama-cpp-python
- CUDA 12.9 source build
- RTX 5090 GPU support
- GGUF chat model support
- SOUL.md persona file
- Basic chatbot loop
- Optional RAG ingestion scaffold
- ChromaDB vector database scaffold
- Windows and WSL startup scripts

2.2 Existing Limitations

- No structured TUI
 - No feature toggles
 - No built-in configuration screen
 - No skill system
 - No automatic memory updates
 - No curator process
 - No task board
 - No model management panel
 - No RAG source viewer
 - No durable session management
 - No safety or tool permission layer
-

3. Goals and Non-Goals

3.1 Goals

The system shall:

1. Provide a terminal-based UI for local chat.
2. Support toggleable RAG.
3. Support toggleable memory.
4. Support toggleable SOUL/persona mode.
5. Support local GGUF models.
6. Support CUDA acceleration.
7. Support persistent user and project memory.
8. Support skill creation and reuse.

9. Support skill pruning through a curator process.
10. Support a simple Kanban/task dashboard.
11. Support local-first operation.
12. Allow users to inspect retrieved RAG context.
13. Allow users to inspect, edit, enable, disable, or delete skills.
14. Avoid unnecessary bloat in prompts.
15. Keep configuration simple and file-based.

3.2 Non-Goals

The system shall not initially:

1. Train or fine-tune models.
2. Replace the local GGUF model weights.
3. Require cloud deployment.
4. Require OpenAI, Anthropic, Grok, or other hosted models.
5. Execute dangerous shell commands without user approval.
6. Automatically expose private files to the model without user-selected RAG folders.
7. Commit local models, vector databases, or virtual environments to Git.

4. System Architecture

4.1 High-Level Architecture

```
Terminal User Interface
|
v
Chat Controller
|
+--> Persona Loader: SOUL.md
+--> Memory Manager: user.md, memory.md, session.md
+--> RAG Retriever: ChromaDB
+--> Skill Manager: skills/
+--> Task Manager: tasks/kanban.json
+--> Model Runtime: llama-cpp-python
+--> Config Manager: config.yaml
```

4.2 Core Modules

4.2.1 TUI Module

Responsible for:

- Chat display
- Input box

- Sidebar panels
- Toggle menu
- RAG source viewer
- Memory viewer
- Skill viewer
- Task board viewer
- Status bar

Recommended libraries:

```
textual
rich
prompt_toolkit
```

Preferred first choice: `textual`.

4.2.2 Model Runtime Module

Responsible for:

- Loading GGUF chat model
- Loading optional embedding model
- Managing context window
- Managing GPU offload settings
- Streaming responses
- Handling chat format selection

4.2.3 Persona Module

Responsible for:

- Loading `SOUL.md`
- Injecting persona into system prompt
- Reloading persona on demand
- Allowing toggle on/off

4.2.4 Memory Module

Responsible for:

- Reading `user.md`
- Reading `memory.md`
- Reading current session summary
- Updating memory every N turns
- Enforcing character limits
- Separating durable memory from temporary session context

4.2.5 RAG Module

Responsible for:

- Ingesting files from `docs/`
- Chunking documents
- Creating embeddings
- Storing vectors in ChromaDB
- Retrieving top K chunks
- Showing retrieved sources in the TUI
- Allowing RAG to be toggled on/off

4.2.6 Skill Module

Responsible for:

- Storing reusable workflows in `skills/`
- Loading skill metadata
- Matching user requests to skills
- Suggesting skill use
- Creating new skills after repeated successful workflows
- Allowing manual skill creation, editing, archiving, and deletion

4.2.7 Curator Module

Responsible for:

- Reviewing stale skills
- Identifying duplicate skills
- Archiving unused skills
- Compacting skill descriptions
- Preventing skill bloat

4.2.8 Task Board Module

Responsible for:

- Maintaining tasks in JSON
 - Providing basic Kanban states
 - Showing tasks in a TUI panel
 - Allowing the assistant to suggest task updates
 - Requiring user approval before modifying tasks
-

5. File and Folder Design

5.1 Recommended Project Structure

```
chatbot-uncensored/  
  app/  
    main.py  
    tui/  
      app.py  
      screens.py  
      widgets.py  
      styles.tcss  
    core/  
      chat_controller.py  
      config.py  
      model_runtime.py  
      prompt_builder.py  
    memory/  
      memory_manager.py  
      user.md  
      memory.md  
      session.md  
    rag/  
      ingest.py  
      retriever.py  
      chunker.py  
      embeddings.py  
    skills/  
      skill_manager.py  
      curator.py  
      registry.json  
      active/  
      archived/  
    tasks/  
      task_manager.py  
      kanban.json  
    utils/  
      logging.py  
      paths.py  
      guards.py  
  docs/  
  models/  
  chromaDb/  
  SOUL.md  
  config.yaml  
  chatbot.py
```

```
ingestDocs.py
start-chatbot.sh
start-chatbot-windows.ps1
README.md
.gitignore
```

5.2 Configuration File

config.yaml

```
model:
  chatModelPath: "./models/NemoMix-Unleashed-12B-Q4_K_M.gguf"
  embeddingModelPath: "./models/embedding-model.gguf"
  contextSize: 8192
  gpuLayers: -1
  threads: 8
  chatFormat: "mistral-instruct"

features:
  soul: true
  rag: true
  memory: true
  skills: false
  curator: false
  kanban: false
  streaming: true
  showSources: true

rag:
  dbPath: "./chromaDb"
  docsPath: "./docs"
  topK: 5
  chunkSize: 1200
  chunkOverlap: 200

memory:
  userFile: "./app/memory/user.md"
  memoryFile: "./app/memory/memory.md"
  sessionFile: "./app/memory/session.md"
  updateEveryTurns: 10
  maxUserChars: 3000
  maxMemoryChars: 6000
  maxSessionChars: 4000

skills:
  activePath: "./app/skills/active"
```

```
archivedPath: "./app/skills/archived"
registryPath: "./app/skills/registry.json"
autoCreate: false
minSuccessfulRepeats: 3
```

```
tasks:
  kanbanPath: "./app/tasks/kanban.json"
```

6. Functional Requirements

6.1 TUI Requirements

TUI-001: Main Chat Interface

The system shall provide a terminal chat interface with:

- Scrollable conversation history
- User input area
- Model response area
- Status bar
- Active feature indicators

TUI-002: Toggle Menu

The system shall allow users to toggle:

- RAG
- Memory
- SOUL.md persona
- Skills
- Curator
- Kanban
- Source display
- Streaming output

TUI-003: Source Viewer

When RAG is enabled, the system shall allow users to inspect retrieved chunks.

TUI-004: Memory Viewer

The system shall allow users to view:

- `user.md`
- `memory.md`

- `session.md`

TUI-005: Skill Viewer

The system shall allow users to list:

- Active skills
- Archived skills
- Suggested skills

TUI-006: Task Board Viewer

The system shall display a simple Kanban board with:

- Backlog
 - In Progress
 - Blocked
 - Done
-

6.2 Chat Requirements

CHAT-001: Local Model Loading

The system shall load a local GGUF model using `llama-cpp-python`.

CHAT-002: GPU Offload

The system shall support GPU offload through `n_gpu_layers`.

CHAT-003: Configurable Chat Format

The system shall allow the user to configure chat format, including:

- `mistral-instruct`
- `chatml`
- `llama-2`
- `llama-3`

CHAT-004: Streaming Responses

The system should support token streaming in the TUI.

CHAT-005: Conversation State

The system shall maintain conversation messages during the active session.

6.3 SOUL.md Requirements

SOUL-001: Persona Loading

The system shall load `SOUL.md` at startup.

SOUL-002: Persona Reload

The system shall allow reloading `SOUL.md` without restarting the app.

SOUL-003: Persona Toggle

The system shall allow persona mode to be enabled or disabled.

SOUL-004: Prompt Injection

When enabled, `SOUL.md` shall be injected as the primary system prompt.

6.4 RAG Requirements

RAG-001: Document Ingestion

The system shall ingest supported files from `docs/`.

RAG-002: Chunking

The system shall split documents into overlapping chunks.

RAG-003: Embedding

The system shall embed document chunks using a local embedding model.

RAG-004: Vector Storage

The system shall store embeddings in ChromaDB.

RAG-005: Retrieval

The system shall retrieve top K relevant chunks for each user query.

RAG-006: Prompt Injection

The system shall inject retrieved context into the model prompt.

RAG-007: Source Display

The system shall show retrieved sources when source display is enabled.

RAG-008: RAG Toggle

The system shall allow RAG to be turned on or off during runtime.

6.5 Memory Requirements

MEM-001: User Memory

The system shall store stable user information in `user.md`.

MEM-002: General Memory

The system shall store durable project and preference memory in `memory.md`.

MEM-003: Session Memory

The system shall store short-term session context in `session.md`.

MEM-004: Character Limits

The system shall enforce strict character limits for memory files.

MEM-005: 10-Turn Memory Nudging

Every 10 turns, the system should summarize important new information and suggest memory updates.

MEM-006: User Approval

The system shall not permanently update memory without user approval unless auto-memory is enabled.

MEM-007: Memory Toggle

The system shall allow memory injection to be toggled on or off.

6.6 Skill System Requirements

SKILL-001: Skill File Format

Each skill shall be stored as a markdown file with metadata.

Example:

```
---
name: build_wsl_cuda_llama_cpp
description: Rebuild llama-cpp-python with CUDA 12.9 for RTX 5090.
tags: [cuda, wsl, llama-cpp-python, gpu]
status: active
successCount: 3
lastUsed: 2026-06-10
---

## Trigger

User needs to rebuild or repair CUDA support for llama-cpp-python.

## Procedure

1. Activate WSL venv.
2. Export CUDA paths.
3. Build with GGML_CUDA.
4. Set CMAKE_CUDA_ARCHITECTURES=120.
5. Test with ldd and testGpu.py.

## Validation

Expected logs include CUDA0 compute buffer and ARCHS = 1200.
```

SKILL-002: Manual Skill Creation

The system shall allow the user to create skills manually.

SKILL-003: Suggested Skill Creation

The system should detect repeated successful workflows and suggest crystallizing them into a skill.

SKILL-004: Skill Invocation

The system shall be able to inject relevant skill instructions into the prompt.

SKILL-005: Skill Toggle

The system shall allow skill usage to be turned on or off.

SKILL-006: Skill Registry

The system shall maintain a registry of skill metadata.

6.7 Curator Requirements

CUR-001: Skill Review

The curator shall review skills based on:

- Last used date
- Success count
- Duplicate overlap
- User rating
- File size
- Relevance

CUR-002: Archive Suggestions

The curator shall suggest archiving stale or duplicate skills.

CUR-003: Compaction

The curator should suggest shorter versions of bloated skills.

CUR-004: Manual Approval

The curator shall not delete skills without user approval.

CUR-005: Curator Toggle

The curator shall be toggleable.

6.8 Kanban Requirements

TASK-001: Task Creation

The system shall allow task creation.

TASK-002: Task Movement

The system shall allow moving tasks between:

- Backlog
- In Progress
- Blocked
- Done

TASK-003: Assistant Suggestions

The assistant may suggest task updates based on conversation.

TASK-004: User Approval

The system shall require approval before modifying the Kanban board.

7. Nonfunctional Requirements

7.1 Performance

- Chat model shall load successfully on local hardware.
- RAG retrieval should complete within 2 seconds for small document sets.
- TUI should remain responsive during generation.
- Streaming should begin as soon as tokens are available.

7.2 Reliability

- Startup scripts should consistently activate WSL and the venv.
- Missing models should produce clear errors.
- Missing embedding models should produce clear errors.
- RAG should fail gracefully if no database exists.
- TUI should not crash from malformed memory or skill files.

7.3 Maintainability

- Features shall be modular.
- Config shall be centralized in `config.yaml`.
- Large files shall not be committed.
- Each module shall have a clear responsibility.

7.4 Privacy

- All chat, memory, and RAG data shall remain local by default.
- No cloud calls shall be made unless explicitly configured.
- Sensitive files shall not be indexed unless placed in allowed folders.

7.5 Safety

- Shell command execution shall be disabled by default.
 - File writing shall require clear module boundaries.
 - Autonomous skill creation shall require user approval in early versions.
 - Curator deletion shall require user approval.
-

8. Iterative Development Plan

Iteration 0: Stabilize Current Local Chatbot

Objective

Lock in the working GPU GGUF chatbot baseline.

Deliverables

- Working `chatbot.py`
- Working `SOUL.md`
- Working CUDA startup script
- Clean `.gitignore`
- README
- `requirements.txt`

Acceptance Criteria

- App starts from PowerShell script.
 - WSL activates `.venv-wsl`.
 - Chat model loads.
 - GPU logs show CUDA usage.
 - `.venv-wsl`, `models`, and `chromaDb` are ignored by Git.
-

Iteration 1: Refactor Into Modules

Objective

Move from one large script to a maintainable app structure.

Deliverables

- `app/main.py`
- `app/core/model_runtime.py`
- `app/core/config.py`
- `app/core/prompt_builder.py`
- `app/memory/memory_manager.py`
- `app/rag/retriever.py`

Acceptance Criteria

- Existing chatbot behavior still works.
- Config values are loaded from `config.yaml`.
- No hardcoded model paths remain in core logic.

- Startup script runs `python -m app.main`.
-

Iteration 2: Add Basic TUI

Objective

Replace plain terminal input loop with a TUI.

Deliverables

- Textual-based TUI app
- Chat history panel
- Input box
- Status bar
- Basic commands

Commands

```
/help  
/exit  
/reload-soul  
/status
```

Acceptance Criteria

- User can chat through TUI.
 - Model responses display in scrollable history.
 - Status bar shows model name and active features.
 - App exits cleanly.
-

Iteration 3: Add Feature Toggles

Objective

Add runtime controls for major systems.

Deliverables

- Toggle menu
- Feature state manager
- Config persistence

Toggles

```
RAG: on/off  
Memory: on/off  
SOUL: on/off  
Skills: on/off  
Curator: on/off  
Kanban: on/off  
Sources: on/off  
Streaming: on/off
```

Acceptance Criteria

- User can toggle features without restarting.
 - Current toggle state is visible.
 - Toggle state can be saved to config.
-

Iteration 4: Add RAG v1

Objective

Add practical document retrieval.

Deliverables

- `docs/` ingestion
- ChromaDB storage
- Embedding model support
- Retrieval before generation
- Source panel

Commands

```
/rag on  
/rag off  
/ingest  
/sources
```

Acceptance Criteria

- Documents can be indexed.
- User questions retrieve relevant chunks.
- Retrieved chunks are injected into prompt.
- Sources can be viewed in the TUI.

Iteration 5: Add Persistent Memory v1

Objective

Add curated local memory files.

Deliverables

- user.md
- memory.md
- session.md
- Memory viewer
- Memory prompt injection
- Manual memory updates

Commands

```
/memory  
/memory-edit  
/memory-on  
/memory-off
```

Acceptance Criteria

- Memory files load at startup.
- Memory is injected only when enabled.
- User can view memory in TUI.
- Memory stays under character limits.

Iteration 6: Add 10-Turn Memory Nudging

Objective

Add real-time memory review.

Deliverables

- Turn counter
- Memory update suggestion prompt
- User approval flow
- Memory compaction

Acceptance Criteria

- Every 10 turns, the system reviews conversation context.
 - It suggests updates to `memory.md` or `user.md`.
 - User can accept, reject, or edit.
 - Memory files remain within limits.
-

Iteration 7: Add Skill System v1

Objective

Add reusable workflow files.

Deliverables

- `skills/active`
- `skills/archived`
- `registry.json`
- Skill loader
- Skill viewer
- Manual skill creation

Commands

```
/skills  
/skill-new  
/skill-view  
/skill-run  
/skill-archive
```

Acceptance Criteria

- Skills load from disk.
 - User can create a skill.
 - User can invoke a skill manually.
 - Skill content can be injected into prompt.
-

Iteration 8: Add Skill Crystallization

Objective

Suggest new skills after repeated successful workflows.

Deliverables

- Workflow observation log
- Success marker
- Skill suggestion prompt
- Skill template generator

Commands

```
/success  
/crystallize
```

Acceptance Criteria

- User can mark a workflow as successful.
 - After repeated success, the app suggests creating a skill.
 - Generated skill includes trigger, procedure, and validation.
 - User approves before saving.
-

Iteration 9: Add Curator v1

Objective

Prevent skill bloat.

Deliverables

- Curator review command
- Duplicate detection
- Stale skill detection
- Archive suggestions

Commands

```
/curator  
/curator-review  
/curator-archive
```

Acceptance Criteria

- Curator lists stale skills.
- Curator suggests duplicate merges.
- Curator does not delete automatically.
- User approves archival.

Iteration 10: Add Kanban Dashboard

Objective

Add task management in the TUI.

Deliverables

- `kanban.json`
- Task board panel
- Task creation
- Task movement
- Assistant task suggestions

Commands

```
/tasks  
/task-new  
/task-move  
/task-done
```

Acceptance Criteria

- User can create tasks.
- User can move tasks between columns.
- Assistant can suggest updates.
- User approves changes.

Iteration 11: Add Session Persistence

Objective

Allow sessions to resume.

Deliverables

- Session save/load
- Conversation summaries
- Session browser

Commands

```
/session-save  
/session-load  
/session-list  
/session-summary
```

Acceptance Criteria

- Conversation can be saved.
 - Previous session can be loaded.
 - Session summary can be injected into context.
-

Iteration 12: Add Stability Layer

Objective

Make the app feel like a product, not a fragile script.

Deliverables

- Health check command
- Startup diagnostics
- Config validator
- Friendly error handling
- Log files

Commands

```
/health  
/diagnostics  
/config
```

Acceptance Criteria

- Missing model path shows useful error.
 - Missing embedding model shows useful error.
 - CUDA setup is checked.
 - RAG database status is checked.
 - App does not crash on common setup errors.
-

Iteration 13: Add Optional Tool Harness

Objective

Create a controlled way for the assistant to interact with the local environment.

Deliverables

- Tool registry
- Permission layer
- Read-only file tools
- Optional write tools
- Optional shell tools

Tool Categories

```
read_file
write_file
list_dir
search_docs
run_command
create_task
update_memory
create_skill
```

Acceptance Criteria

- Tools are disabled by default.
 - Risky tools require approval.
 - Tool usage is logged.
 - TUI shows tool calls clearly.
-

Iteration 14: Polish and Packaging

Objective

Make setup and usage clean.

Deliverables

- Better README
- requirements.txt
- setup.sh
- doctor.sh
- Example config

- Example docs
- Example SOUL.md
- Example skills

Acceptance Criteria

- New user can set up the app from README.
 - `doctor.sh` detects common issues.
 - Startup script works from Windows.
 - Project can be cloned without large local files.
-

9. Suggested Commands

Core Commands

```
/help  
/exit  
/status  
/reload  
/config  
/health
```

Persona Commands

```
/soul  
/soul-on  
/soul-off  
/reload-soul
```

RAG Commands

```
/rag  
/rag-on  
/rag-off  
/ingest  
/sources
```

Memory Commands

```
/memory  
/memory-on  
/memory-off
```



```
/memory-review  
/memory-accept  
/memory-reject
```

Skill Commands

```
/skills  
/skill-new  
/skill-run  
/skill-view  
/skill-archive  
/crystallize
```

Curator Commands

```
/curator  
/curator-review  
/curator-archive  
/curator-compact
```

Task Commands

```
/tasks  
/task-new  
/task-move  
/task-done  
/task-delete
```

10. Prompt Assembly Order

When all features are enabled, the system shall assemble prompts in this order:

1. Core system rules
2. SOUL.md
3. user.md
4. memory.md
5. session.md summary
6. Relevant skills
7. Retrieved RAG context
8. Current user message

The prompt builder shall respect feature toggles.

If RAG is disabled, skip retrieved context.

If memory is disabled, skip memory files.

If skills are disabled, skip skill injection.

11. Feature Toggle State Example

```
{
  "soul": true,
  "rag": true,
  "memory": true,
  "skills": false,
  "curator": false,
  "kanban": false,
  "sources": true,
  "streaming": true
}
```

12. Acceptance Test Matrix

Feature	Test	Expected Result
Startup	Run PowerShell script	WSL opens and chatbot starts
CUDA	Run <code>/health</code>	CUDA detected
Chat	Send message	Model responds
SOUL	Edit SOUL.md and reload	Persona changes
RAG	Ask about indexed doc	Answer cites retrieved context
RAG Toggle	Disable RAG	No context retrieved
Memory	View memory	Memory files display
10-Turn Nudge	Chat 10 turns	Memory review triggers
Skills	Create skill	Skill appears in registry
Curator	Run review	Stale skills listed
Kanban	Create task	Task appears in backlog

Feature	Test	Expected Result
Git Hygiene	Run git status	No models, venv, or DB staged

13. Risks

13.1 Prompt Bloat

Risk: Too much memory, RAG, and skill content may overflow context.

Mitigation:

- Character limits
- Top K retrieval
- Skill relevance filtering
- Memory compaction

13.2 Bad Skill Creation

Risk: The system may create low-quality or redundant skills.

Mitigation:

- Require user approval
- Add curator
- Track success count
- Archive stale skills

13.3 RAG Hallucination

Risk: Model may answer beyond retrieved context.

Mitigation:

- Clear RAG prompt
- Source viewer
- "Not enough context" instruction
- Optional source citations

13.4 Fragile CUDA Setup

Risk: CUDA setup may break after updates.

Mitigation:

- Document known working build

- Add `/health`
- Store build notes
- Keep startup scripts simple

13.5 Unsafe Tool Use

Risk: Agent may perform unwanted local actions.

Mitigation:

- Tools off by default
- Approval gates
- Logs
- Read-only mode

14. Minimum Viable Product

The MVP shall include:

1. TUI chat.
2. SOUL.md loading.
3. Config file.
4. RAG ingestion and retrieval.
5. Feature toggles.
6. Source viewer.
7. Basic memory files.
8. Startup diagnostics.

The MVP does not require:

- Auto skill creation
- Curator
- Kanban
- Tool execution
- Cloud deployment

15. Version Roadmap

v0.1

Stable GPU local chatbot.

v0.2

Modular app refactor.

v0.3

TUI interface.

v0.4

RAG ingestion and retrieval.

v0.5

Feature toggles and source viewer.

v0.6

Persistent memory files.

v0.7

10-turn memory nudging.

v0.8

Manual skill system.

v0.9

Skill crystallization.

v1.0

Stable TUI RAG chatbot product.

v1.1

Curator.

v1.2

Kanban dashboard.

v1.3

Controlled tool harness.

16. Definition of Done

The project reaches v1.0 when:

- It starts reliably from Windows.
- It runs the local GGUF model on GPU.
- It provides a usable TUI.
- It supports RAG with visible sources.
- It supports `SOUL.md`.
- It supports memory files with limits.
- It supports feature toggles.
- It has clean Git hygiene.
- It has clear error messages.
- It can be used daily without manual debugging.